

UC LINA 600 LINEAR ALGEBRA FOR GRAPHICS · MODULE 08 · LECTURE NOTES

The Pipeline as a Matrix Chain

Where the course pays off, visibly.

MODULE OUTCOMES

- Derive the model and view matrices.
- Derive a perspective projection matrix.
- Explain the depth mapping and its precision problem.
- Explain the viewport transform.
- Assemble and verify the full chain.

1 Model and view

THE MODEL MATRIX COMPOSES THE OBJECT'S TRANSFORMS AND THE VIEW MATRIX IS THE CAMERA FRAME INVERTED

Model: scale, then rotate, then translate — composed as $T \cdot R \cdot S$ and applied right to left (Module 03 §2), which is the conventional order and gives the behaviour people expect: the object scales about its own origin, turns about its own origin, and then moves into place.

And the order matters visibly: rotating after translating makes the object orbit the world origin instead of spinning in place — which is occasionally what you want and is usually a bug, and is instantly recognisable once you have seen it.

View: build the camera's orthonormal frame in world coordinates and invert it (Module 04 §3) — which, because the frame is orthonormal, is a transpose plus a negated translation, and is about four lines of code.

So two of the pipeline's four matrices are already yours, from Modules 03 and 04 — and the projection is the only genuinely new one, which is §2.

2 Deriving the projection

THE GEOMETRY: a point at depth z projects onto a screen plane at distance d by similar triangles:

$$\begin{aligned}x_{\text{screen}} &= d * x / (-z) && (z \text{ is negative;} \\y_{\text{screen}} &= d * y / (-z) && \text{camera looks } -z)\end{aligned}$$

THAT IS A DIVISION, so we need w (Module 07):
arrange for $w = -z$ and let the hardware divide.

$$\begin{aligned}\begin{bmatrix} d & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & A & B \\ 0 & 0 & -1 & 0 \end{bmatrix} & \text{ so } \begin{aligned}x' &= d*x \\ y' &= d*y \\ z' &= A*z + B \\ w' &= -z\end{aligned}\end{aligned}$$

after the divide: $x'/w' = -d*x/z$. Correct.

AND A , B ARE CHOSEN to map the near and far planes to whatever depth range the hardware wants:

$$\begin{aligned}z = -n & \rightarrow z'/w' = -1 \quad (\text{or } 0) \\ z = -f & \rightarrow z'/w' = +1\end{aligned}$$

two equations, two unknowns, and you solve them.

THAT IS THE WHOLE DERIVATION.

Similar triangles, one row to put the depth into w , and two equations to fix the depth range — which is the whole projection matrix, and takes about ten minutes the first time. **Project 2 requires this derived rather than copied**, because the derivation is what makes §3's consequence visible, and because a copied matrix in the wrong depth convention produces a scene that is entirely black for reasons nothing explains.

3 Depth precision

DEPTH AFTER THE DIVIDE IS NON-LINEAR IN Z , SO ALMOST ALL THE PRECISION SITS NEAR THE NEAR PLANE

z'/w' works out to something of the form $a + b/z$ — a reciprocal in the depth — so equal steps in the stored depth value are emphatically not equal steps in actual distance, and the relationship is steep.

Which means a near plane at 0.01 and a far plane at 10,000 spends the great majority of the depth buffer's precision on the first metre of the scene — and everything beyond that fights over the few values left.

That is z -fighting: two distant surfaces rounding to the same stored depth and flickering between frames as the camera moves — and the fix is almost always to move the near plane further out, not to pull the far plane in, because the ratio f/n is what governs the loss and n is the sensitive term.

Which is a specific, extremely common, and frequently misdiagnosed bug — and it falls directly out of §2's derivation, which is precisely why deriving rather than copying matters (Module 01 §1's argument, now concrete).

4 Assembling and checking

- $p_clip = Projection \cdot View \cdot Model \cdot p_model$ — applied right to left — and the from-naming convention makes the composition checkable by inspection (Module 04 §2: $clip_from_camera \cdot camera_from_world \cdot world_from_model$).
- Then the divide by w , which produces normalised device coordinates inside a cube — and **anything falling outside that cube is clipped** before rasterisation.
- Then the viewport transform: scale and shift the cube to pixel coordinates, which is a trivial affine map — and is where the y-flip usually hides, since screen y conventionally grows downward and device y grows upward.
- And check each stage separately: take a point whose position you computed by hand, and verify it at every stage of the chain rather than looking at the final image (Module 13 §2's verification suite).
- Which is the only practical way to localise an error in a chain of four matrices — the image tells you that something is wrong and never which matrix, and bisecting the chain by hand is the whole technique.

AND THE COURSE'S PAYOFF IS HERE

Everything in Modules 02 through 07 was for this — bases, change of basis, linearity, and homogeneous coordinates — all four used in a single derivation, which is a satisfying thing to notice.

And the whole derivation is about twenty lines — which is why this course is four weeks rather than a semester, and why skipping it costs more than it saves.

Plus: you can now debug it, which was Module 01 §1's entire argument — a copied matrix gives you a picture and a derived one gives you a diagnosis, and §3's z-fighting is the clearest example.

So Project 2 asks for the derivations alongside the working renderer — because the renderer alone does not demonstrate the thing this course was for, and a renderer built from copied matrices looks identical to one built from understood ones until the first bug.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Scratchapixel, the perspective projection series (free)	§§2 and 3 — the derivation done slowly and in full, free, with code.
Gortler, chapters 10 and 11 (library copy)	§§1 and 2 — the pipeline with every frame named, which is the careful version.

Resource	What it covers for this module
Song Ho Ahn — OpenGL projection matrix (free)	§2 — the standard derivation with every algebraic step shown, which is useful for checking yours.
Reversed-Z and depth precision writeups (free)	§3 — depth precision visualised, and the reversed-Z technique that largely solves it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Build a model matrix** as TRS and verify the composition order.
- 2 **Swap the order** and describe what the object does.
- 3 **Build a view matrix** from a camera position and target.
- 4 **Derive the projection matrix** from similar triangles, yourself.
- 5 **Solve for A and B** given a near and far plane.
- 6 **Plot stored depth against distance** and observe the reciprocal.
- 7 **Produce z-fighting deliberately**, then fix it by moving the near plane.
- 8 **Build the viewport transform**, including the y-flip.
- 9 **Trace one point through all four stages** by hand.
- 10 **Verify your implementation** against those hand numbers.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the model matrix's composition order and why.
- 2 What happens if you rotate after translating?
- 3 How is the view matrix built, and why is it an inverse?
- 4 Derive the projection matrix from similar triangles.
- 5 What are A and B for, and how are they found?
- 6 Why is stored depth non-linear in distance?
- 7 What is z-fighting and what is the usual fix?
- 8 Write the full chain, right to left.
- 9 Where does the y-flip live?

10 How do you localise an error in a four-matrix chain?